

User Authentication Flow

Overview

The user authentication process ensures that user credentials are transmitted securely and verified using Spring Security. The application uses **RSA encryption** to protect the password during transmission and **BCrypt hashing** to verify the password stored in the database. Additionally, security mechanisms such as **Multi-Factor Authentication (MFA)** and **Account Locking** are implemented to enhance protection against unauthorized access.

Step 1: User Login

The authentication process begins when the user enters their **Username** and **Password** on the application's login page and clicks the **Login** button.

At this stage, the password exists only within the browser and has not yet been transmitted to the server.

Step 2: Clear Existing Authentication Cookies

Before initiating a new authentication session, the frontend invokes the following API:

POST /api/v1/auth/clearcookies

Purpose

This API removes any authentication cookies or session data from previous login sessions.

Benefits

- Prevents conflicts with previous login sessions.
 - Ensures a fresh authentication process.
 - Eliminates stale authentication tokens.
 - Improves session security.
-

Step 3: Fetch RSA Public Key

The frontend then requests the RSA public key from the backend.

GET /api/v1/auth/public-key

Purpose

The backend generates and returns an **RSA Public Key**.

The public key appears as a long encoded string and is not human-readable because it is generated using the RSA encryption algorithm.

Example:

```
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQE...
```

Why is the Public Key Required?

The frontend uses this public key to encrypt the user's password before sending it to the server.

This ensures that:

- The password is never transmitted in plain text.
- Even if network traffic is intercepted, the password cannot be read.
- Only the server can decrypt the encrypted password because it possesses the matching RSA Private Key.

Step 4: Encrypt Password and Send Login Request

Once the frontend receives the RSA public key:

1. The password is encrypted using RSA.
2. The encrypted password is included in the authentication request.
3. The request is sent to:

```
POST /auth/authenticate
```

The request typically contains:

- Username
- Encrypted Password

At this stage, the original password is never exposed over the network.

Step 5: Authentication Processing

The authentication request is received by the backend.

The request is processed inside the **processAuthentication()** method.

The first validation performed is checking whether **Multi-Factor Authentication (MFA)** is enabled for the user.

Example:

```
if (isMultiFactorAuthenticationEnabled()) {  
    // Perform MFA validation  
}
```

Purpose

If MFA is enabled, the user must complete the additional verification step before authentication is completed.

This provides an additional layer of security.

Step 6: Decrypt the Password

Since the password arrives in encrypted form, the backend must decrypt it before authentication.

The backend uses:

- RSA Private Key
- Java Cipher API

Example:

```
Cipher cipher = Cipher.getInstance("RSA");  
cipher.init(Cipher.DECRYPT_MODE, privateKey);  
  
String decryptedPassword = new String(  
    cipher.doFinal(encryptedPasswordBytes)  
);
```

Result

The decrypted password exists only temporarily in the server's memory.

It is never stored in plain text.

Step 7: Authenticate Using Spring Security

After decryption, Spring Security's **AuthenticationManager** validates the user's credentials.

Example:

```
authenticationManager.authenticate(  
    new UsernamePasswordAuthenticationToken(  
        userName,  
        decryptedPassword  
    )  
);
```

During authentication, Spring Security performs the following validations:

- Retrieves the user details from the database.
- Retrieves the BCrypt hashed password stored in the database.
- Compares the entered password with the stored BCrypt hash.
- Checks whether the account is enabled.
- Checks whether the account is active.
- Checks whether the account is locked.
- Checks whether the account has expired.
- Checks whether the user's credentials have expired.

If all validations succeed, authentication is successful.

Step 8: Invalid Credentials Handling

If the username or password is incorrect, Spring Security throws a **BadCredentialsException**.

Example:

```
throw new BadCredentialsException("Invalid Credentials");
```

The backend returns an authentication failure response.

The frontend displays the following message:

```
Invalid Credentials
```

No authentication token or session is created.

Step 9: Account Locking

To protect against brute-force attacks, the application tracks consecutive failed login attempts.

For example:

- First failed attempt
- Second failed attempt
- Third failed attempt
- Fourth failed attempt
- Fifth failed attempt

After reaching the configured maximum number of failed attempts, the user's account is automatically locked.

When the Account is Locked

- The user cannot log in, even with the correct password.
- Authentication requests are rejected.
- The application informs the user that the account has been locked.
- The user must contact the Organization Administrator to unlock the account.

Benefits

- Prevents brute-force attacks.
- Protects user accounts from repeated unauthorized login attempts.
- Improves overall application security.

Complete Authentication Flow

User

|



Enter Username & Password

|



Clear Existing Cookies

|



Request RSA Public Key

|



Receive RSA Public Key

|



Encrypt Password Using RSA

|



POST /auth/authenticate

|



Backend Receives Request

|



Check MFA Configuration

|



Decrypt Password Using RSA Private Key

|



AuthenticationManager.authenticate()

|



Retrieve User Details

|



Compare Password with BCrypt Hash

|



Check Account Status

|

|—————→ Invalid Password

|

|

|



|

Increment Failed Attempts

|

|

|



|

Lock Account After Limit

|



Authentication Successful



Generate Session / JWT / Authentication Token



Summary

The application follows a secure authentication mechanism by combining **RSA encryption, Spring Security, BCrypt password hashing, Multi-Factor Authentication (MFA), and Account Locking**. Before authentication begins, previous authentication cookies are cleared to ensure a fresh session. The user's password is encrypted on the client side using an RSA public key and decrypted only on the server using the corresponding private key. Spring Security then validates the decrypted password against the BCrypt hash stored in the database while also checking the account status. If authentication succeeds, the user is granted access. If repeated authentication failures occur, the account is automatically locked to protect against brute-force attacks.